# Deepfake Detection System PRO

**Comprehensive Project Documentation**

---

## 1. Project Overview

The **Deepfake Detection System PRO** is an advanced, hybrid desktop and web application designed to identify manipulated media—including images and videos—using state-of-the-art Machine Learning models. The system is engineered to provide local, high-speed, and secure inference, minimizing reliance on external APIs to ensure that sensitive media remains confidential.

The application features a dark-themed, highly responsive "glassmorphism" interface built with HTML/CSS/JS, driven by both a Python (Flask) backend and an integrated client-side ONNX Runtime environment for unparalleled speed.

## 2. Core Features

- **Dual-Inference Pipeline:** Utilizes both a backend **PyTorch / OpenCV** engine and a frontend **ONNX.js + Face-API.js** engine. This ensures the system can gracefully fallback and cross-verify results.
- **Real-time Confidence Metrics:** Calculates and visually represents the probability of media being `Authentic (Real)` versus `Manipulated (Fake)` using dynamic Gauge charts.
- **History & Auditing:** Automatically saves all scan results into browser storage ( `localStorage` ) and syncs with the `/api/history` backend endpoint, allowing users to track previous detections securely.
- **Advanced Comparison Mode (A/B Test):** Users can upload a reference image alongside a suspected fake to run isolated difference checks, outputting a clear verdict on manipulation variations.
- **Heatmap & Visual Transparency:** Through integration with Face-API, the tool captures facial bounding boxes and paints localized heatmap indicators over detected manipulations (e.g., face-swapping boundaries).
- **Data Export Capabilities:** Users can physically export their inference reports securely as **PDF Documents** or **JSON formatted text files** directly from the UI.
- **Confidentiality-First Design:** The tool processes inputs largely locally ( `deepfake_v2.onnx` ), meaning highly sensitive corporate or journalistic media never needs to leave the machine's memory boundaries.

## 3. How It Works (The Pipeline)

When a user uploads a file, the system triggers a 4-step pipeline:

### Step 1: File Validation

Validates the file format (JPG, PNG, MP4, AVI, MOV). For immediate UI feedback, a local ObjectURL is generated to securely display the preview completely offline.

### Step 2: Face Detection & Extraction

If processing an image, the system utilizes `face-api.js` (loaded via CDN or strictly cached Weights). It maps the biological landmarks of subjects inside the frame. If a face is found, it dynamically crops the facial region to eliminate background noise, maximizing the deepfake model's focus.

### Step 3: Deepfake ML Scan

The system invokes the ONNX Inference Session using the proprietary `deepfake_v2.onnx` model.

1. **Preprocessing:** The cropped face is normalized (standardized using RGB mean/std) and reshaped into a Float32 Tensor structured as `1x3x224x224`.
2. **Model Execution:** The ONNX Runtime processes the tensor, returning logits for `[fake, real]`.
3. **Heuristics & Softmax:** The raw logits are passed through a Softmax mathematical function to convert them into readable confidence percentages (e.g., 97.4%). Feature artifacts (like GAN fingerprints and Compression artifacts) are additionally analyzed.

### Step 4: Results & Reporting

The processed values update the user interface immediately. If confidence drops below 60%, a Yellow "Uncertainty" badge warns the user. If Fake confidence is extremely high (>85%), a Red Critical warning is fired. Simultaneously, the history log is appended, updating the global Dashboard Statistics.

## 4. Technical Architecture

### Frontend (Browser UI)

- **Technologies:** Vanilla JavaScript (ES6+), HTML5, CSS3.
- **Layout:** Responsive CSS Grid / Flexbox setup with Media Queries accommodating desktop laptops down to mobile device widths.
- **ML Libraries:**
  - `onnxruntime-web` (Client-side Model Execution)
  - `face-api.js` (Facial extraction)
  - `html2pdf.js` (Document Generation)

### Backend (Server)

- **Technologies:** Python 3, Flask Web Framework.
- **Dependencies:** `torch`, `torchvision`, `opencv-python-headless`, `librosa`.
- **Endpoints:**
  - `POST /api/detect`: Primary external handling route for server-bound verifications.
  - `GET /api/history`: Handles synchronized database queries representing total lifecycle scans.

## 5. Setup & Running Instructions

To run this project from a cold start, two separate processes must be deployed in tandem.

**1. Activating the Backend:** Inside terminal 1 (Project Root directory):

1. Verify Python 3 is installed.
2. Activate the virtual environment: `.\venv\Scripts\activate`
3. Boot the application: `python app.py`

**2. Activating the Frontend:** Inside terminal 2:

1. Utilize Python's HTTP host to bypass Cross-Origin restrictions on ONNX fetch operations: `python -m http.server 8000`
2. Open your browser and navigate to exactly: `http://localhost:8000`

## 6. Future Enhancements & Scalability

While presently functional, the codebase is primed for upcoming modules:

- **Audio/Voice Deepfake Detection:** The backend is prepped with `librosa` and `soundfile` dependencies. Expanding the frontend pipeline to accept `WAV` / `MP3` to scan for synthetically cloned voices.
- **Database Integration:** Upgrading the application to securely persist large histories into an SQL database via `flask-sqlalchemy` rather than localized `localStorage` arrays.

---

*Document Generated automatically for System Operators.*